

Capítulo 14

Manutenção e Evolução de Software

Este capítulo apresenta os conceitos de *manutenção e evolução de software*, iniciando pelas *Leis de Lehman* (Seção 14.1), que procuram explicar a necessidade e as características do processo de manutenção e evolução. Em seguida são caracterizadas as quatro *formas de manutenção em função dos objetivos* (Seção 14.2). O *processo de manutenção* também é explicado (Seção 14.3). Finalmente são apresentados os diferentes tipos de *atividades* de manutenção e suas *métricas* (Seção 14.4), além de *modelos para estimativa de esforço* em atividades de manutenção e evolução (Seção 14.5). O capítulo termina com a apresentação dos conceitos relacionados a *reengenharia e engenharia reversa* (Seção 14.6), que são técnicas frequentemente usadas em processos de manutenção de software.

Manutenção de software é como se denomina, em geral, o processo de adaptação e otimização de um software já desenvolvido, bem como a correção de defeitos que ele possa ter. A manutenção é necessária para que um produto de software preserve sua qualidade ao longo do tempo, pois se isso não for feito haverá uma deterioração do valor percebido desse software e, portanto, de sua qualidade.

Classicamente, a atividade de manutenção é considerada o conjunto de modificações que o software sofre depois de ter sido terminado, ou seja, após o final da fase de transição no UP. Porém, o EUP (Seção 5.4) e o OUM (Seção 5.6) adicionam ao Processo Unificado uma fase chamada “Produção”, que ocorre justamente no momento em que o software está em operação e necessita de manutenção. Assim, as atividades de manutenção são consideradas tão relevantes e necessárias quanto as atividades de desenvolvimento para o sucesso de um produto.

Modernamente, a expressão “manutenção de software” vem sendo substituída ou utilizada em conjunto com “evolução de software”. *Evolução* talvez seja um termo mais adequado, porque as atividades de modificação do software na fase de produção não visam mantê-lo como está, mas fazê-lo evoluir de forma a adaptar-se a novos requisitos ou ainda a corrigir possíveis defeitos.

Em uma interpretação do termo, pode-se considerar “manutenção” o conjunto de tarefas individuais de modificação de um software em uso. Nesse caso, a “evolução” seria o processo de mais longo prazo, ou seja, a evolução do software pode ser vista como uma sequência de manutenções ao longo do tempo.

De outro lado, há autores (Erdil, Finn, Keating, Meattle, Park & Yoon, 2003) que acreditam que apenas as correções de erros (manutenção corretiva) podem ser consideradas atividades tradicionais de manutenção, enquanto a otimização, a adaptação e a prevenção de erros são consideradas evolução. Essa interpretação será seguida neste livro.

14.1 Necessidade de Manutenção e Evolução de Software

Considera-se que, uma vez desenvolvido, um software terá um valor necessariamente decrescente com o passar do tempo. Isso ocorre porque:

- Falhas são descobertas.
- Requisitos mudam.
- Produtos menos complexos, mais eficientes ou tecnologicamente mais avançados são

disponibilizados.

Dessa forma, torna-se imperativo que, simetricamente, para manter o valor percebido de um sistema:

- Falhas sejam corrigidas.
- Novos requisitos sejam acomodados.
- Sejam buscadas simplicidade, eficiência e atualização tecnológica.

A realização dessas atividades é parte do processo de manutenção e evolução de software.

As *Leis de Lehman* (Lehman M. M., 1980; Lehman & J. F. Ramil, 1997) procuram explicar a necessidade e a inevitabilidade da evolução de software. São atualmente oito leis, baseadas nas observações do autor sobre os processos de evolução de sistemas. Elas foram apresentadas em várias publicações entre 1974 e 1996, até chegarem à forma atual.

Inicialmente, Lehman identifica três tipos de sistema:

- *Tipo-S* (especificado): sistemas especificados formalmente e entendidos como objetos matemáticos, cuja correção em relação a uma especificação pode ser provada por ferramentas formais. Um exemplo seria uma biblioteca de funções matemáticas.
- *Tipo-P* (resolução de problemas): sistemas desenvolvidos especificamente para executar um conjunto de procedimentos que por si definem o que o programa pode e deve fazer, como por exemplo, um sistema para jogar xadrez.
- *Tipo-E* (evolucionário): sistemas desenvolvidos pelos processos usuais de análise, projeto e codificação, que têm uso corrente em um ambiente real, isto é, são tipicamente sistemas de informação e outros sistemas não gerados por métodos formais. Um exemplo seria um sistema de folha de pagamento em uma empresa.

Entende-se, então, que os sistemas do tipo-S e tipo-P não são susceptíveis às leis da manutenção de software de Lehman, pois apenas defeitos relacionados à especificação ou procedimentos ausentes podem ocorrer.

Já os sistemas de tipo-E são propensos a defeitos em seu processo de produção. Além disso, pelo fato de serem usados em conexão com o mundo real, esses sistemas podem precisar evoluir para atender a requisitos que mudam conforme o contexto de uso, por exemplo, no caso de legislação que muda, ou objetivos da empresa, ou ainda expectativas dos usuários.

As Leis de Evolução de Lehman, portanto, se aplicam apenas aos sistemas de tipo-E. Elas estabelecem que a evolução de software desse tipo é inevitável, e não apenas resultado de uma equipe de desenvolvimento incapaz. Além disso, as leis mostram que existem limites em relação ao que as equipes de manutenção podem fazer. A autorregulação impede que trabalho demasiado ou insuficiente seja executado, sob pena de descontinuação do sistema.

14.1.1 LEI DA MUDANÇA CONTÍNUA

A primeira lei, de 1974, afirma que *um sistema que é efetivamente usado deve ser continuamente melhorado, caso contrário torna-se cada vez menos útil*, pois seu contexto de uso evolui. Se o programa não evoluir, terá cada vez menos valor até que se chegue à conclusão de que vale a pena substituí-lo por outro.

Essa lei expressa o fato conhecido de que programas suficientemente grandes nunca são terminados. Eles simplesmente continuam a evoluir.

Ela também expressa um princípio segundo o qual o software envelhece como se fosse um organismo vivo. Esse envelhecimento seria resultado das inconsistências entre o software e o ambiente no qual ele está inserido. No caso, o ambiente muda, mas o software não se adapta sozinho. Assim, um processo de evolução é necessário para evitar a obsolescência do software, prolongando

sua vida útil. Caso tal evolução não seja feita, o software pode chegar a ser desativado.

14.1.2 LEI DA COMPLEXIDADE CRESCENTE

A segunda lei, também de 1974, expressa que, *à medida que um programa evolui, sua complexidade inerente aumenta, porque as correções feitas podem deteriorar sua organização interna; isso só não acontece quando medidas específicas de cuidado são tomadas durante as atividades de evolução*, como a refatoração do sistema quando necessário.

Segundo essa lei, à medida que mudanças são introduzidas no software, as interações entre elementos – nem sempre previstas ou planejadas na estrutura do software – fazem que a entropia interna aumente, ou seja, ocorre um crescimento cada vez mais desestruturado.

A cada nova mudança, a estrutura interna do software se torna menos organizada, aumentando gradativamente o custo de manutenções posteriores. De fato, chega-se a um ponto em que a refatoração do sistema se torna obrigatória.

14.1.3 LEI FUNDAMENTAL DA EVOLUÇÃO DE PROGRAMAS: AUTORREGULAÇÃO

A terceira lei, também de 1974, estabelece que *a evolução de programas está sujeita a uma dinâmica de autorregulação controlada por feedback* que faz que as medidas globais de esforço e outros atributos de processo sejam estatisticamente previsíveis (distribuição normal).

Essa lei reconhece que o desenvolvimento e a manutenção de um sistema ocorrem dentro de uma organização com objetivos que vão muito além do sistema. Então, os processos dessa organização acabam regulando a aplicação de esforço a cada um de seus sistemas.

Os pontos de controle e realimentação são exemplos de formas de regulação da organização. Quaisquer processos que fujam muito ao padrão da organização são logo refatorados para se adequar, de forma que o esforço gasto nas diferentes atividades permaneça distribuído de forma normal.

14.1.4 LEI DA CONSERVAÇÃO DA ESTABILIDADE ORGANIZACIONAL: TAXA DE TRABALHO INVARIANTE

A quarta lei, de 1980, expressa que *a taxa média efetiva de trabalho global em um sistema em evolução é invariante no tempo*, isto é, ela não aumenta nem diminui.

Essa lei parece ser a menos intuitiva de todas, pois se acredita que a carga de trabalho aplicada a um projeto depende apenas de decisões da gerência e que vá diminuindo com o tempo. Na prática, porém, as demandas de usuários também influenciam essas decisões, que se mantêm praticamente constantes no tempo.

14.1.5 LEI DA CONSERVAÇÃO DA FAMILIARIDADE: COMPLEXIDADE PERCEBIDA

A quinta lei, também de 1980, expressa que, em geral, *a taxa de crescimento de um sistema é limitada pela necessidade de manter a familiaridade*. Isso ocorre porque, para que um sistema evolua de forma saudável, todos os agentes relacionados a ele devem manter a familiaridade com suas características e funções. Se o sistema crescer demais, essa familiaridade será perdida, e leva tempo para recuperá-la.

Essa lei expressa o fato de que a taxa de crescimento de um sistema é limitada pela capacidade dos indivíduos envolvidos em absorver as novidades coletiva e individualmente. Os dados observados sugerem que, se certo valor-limite de novidades for excedido, mudanças comportamentais ocorrerão de forma a baixar a taxa de crescimento.

14.1.6 LEI DO CRESCIMENTO CONTÍNUO

A sexta lei, de 1980, estabelece que *o conteúdo funcional de um sistema deve crescer continuamente para manter a satisfação do usuário*.

Essa lei estabelece ainda que a mudança no software será sempre necessária, seja pela correção de erros (manutenção corretiva), aperfeiçoamento de funções existentes (manutenção perfectiva), seja pela adaptação a novos contextos (manutenção adaptativa).

14.1.7 LEI DA QUALIDADE DECRESCENTE

A sétima lei, de 1996, expressa que *a qualidade de um sistema vai parecer diminuir com o tempo, a não ser que medidas rigorosas sejam tomadas para mantê-lo e adaptá-lo*.

Essa lei estabelece que, mesmo que um software funcione perfeitamente por muitos anos, isso não significa que continuará sendo satisfatório. Conforme o tempo passa, os usuários ficam mais exigentes em relação ao software e, conseqüentemente, mais insatisfeitos com ele.

14.1.8 LEI DO SISTEMA REALIMENTADO

A oitava lei, de 1996, estabelece que *a evolução de sistemas é um processo de realimentação multinível, multiloop e multiagente*, devendo ser encarado dessa forma para que se obtenham melhorias significativas em uma base razoável.

Essa lei lembra que a evolução de software é um sistema retroalimentado e complexo que recebe *feedback* constante dos vários interessados. Em longo prazo, a taxa de evolução de um sistema acaba sendo determinada pelos retornos positivos e negativos de seus usuários, bem como pela quantidade de verba disponível, pelo número de usuários solicitando novas funções, interesses administrativos etc.

14.1.9 VALIDAÇÃO EMPÍRICA DAS LEIS DE LEHMAN

Vários estudos, ao longo das últimas décadas têm colocado as Leis de Lehman em experimentação. **Herraiz, et al. (2013)** publicaram um estudo bibliográfico no qual consolidam os resultados de vários estudos empíricos anteriores. Em geral, eles verificaram que o grupo de sistemas do tipo-E ainda é bastante extenso e heterogêneo e nem sempre as leis valem para qualquer produto neste grupo.

Em especial, os autores procuraram verificar se as Leis se aplicavam a software livre, ou seja, sistemas que são evoluídos por uma comunidade. Em resumo, eles chegaram às seguintes conclusões:

- A primeira lei (mudança contínua) se aplica bem a sistemas livres porque projetos de sucesso são continuamente evoluídos ao longo do tempo. Porém, em algumas ocasiões, mesmo estes projetos podem experimentar períodos de falta de atividade.
- A quarta lei (crescimento contínuo) também foi confirmada. Embora as vezes até experimentem períodos de pouco ou nenhum crescimento, projetos de software livre de sucesso aumentam sua funcionalidade continuamente.
- A sétima lei (qualidade decrescente) é mais difícil de avaliar porque depende de como se mede qualidade. Mas foi observado que o número de defeitos encontrados em software livre ao longo do tempo tende a crescer, o que pode estar confirmando essa lei.
- A oitava lei (sistema realimentado) foi confirmada. Mas foi observado que no caso de software livre, as interações de feedback tendem a ser muito mais caóticas do que no caso de sistemas desenvolvidos em uma única organização.

As demais leis não foram validadas no estudo, mas também não foram invalidadas e, assim,

devem ser objeto de trabalhos futuros.

14.2 Classificação das Atividades de Manutenção

Segundo a Norma ISO/IEC 14764:2006 a manutenção ou evolução de software deve ser classificada em quatro tipos:

- *Corretiva*: toda atividade de manutenção que visa corrigir erros ou defeitos do software.
- *Adaptativa*: toda atividade que visa adaptar as características do software a requisitos que mudaram, sejam novas funções, sejam questões tecnológicas.
- *Perfectiva*: toda atividade que visa melhorar o desempenho ou outras qualidades do software sem alterar necessariamente sua funcionalidade.
- *Preventiva*: toda atividade que visa melhorar as qualidades do software de forma que erros potenciais sejam descobertos e mais facilmente resolvidos antes que se tornem problemas maiores.

As subseções seguintes discorrem sobre cada um destes tipos.

14.2.1 MANUTENÇÃO CORRETIVA

A *manutenção corretiva* visa corrigir possíveis defeitos do software que provocam erros. Suas atividades podem ainda ser subdivididas em dois subgrupos:

- Manutenção para correção de *erros conhecidos*.
- Manutenção para detecção e correção de *novos erros*.

Os erros conhecidos de um software costumam ser registrados em um documento de considerações operacionais ou em notas de versão ([Seção 10.3](#)), de forma que os usuários do software possam contornar as falhas e evitar maiores transtornos. São aqueles erros que já foram identificados, mas que não foram corrigidos ou porque o processo de depuração ainda não localizou a causa do erro, ou porque as atividades de correção seriam tão longas e complexas que é preferível ter o sistema funcionando, mesmo com o erro, do que paralisar o uso do sistema até que o defeito possa ser corrigido.

Conforme visto no [Capítulo 13](#), nem sempre o fato de se saber que um software tem um erro implica saber onde se localiza o defeito que provoca esse erro. Assim, a atividade de teste pode até ter descoberto que, em determinada situação, o software não se comporta de acordo com o esperado, mas a atividade de depuração pode não ter ainda encontrado o ponto no código-fonte causador desse erro. Nesse caso, enquanto a atividade de depuração ou mesmo de refatoração do software prossegue, a versão atual do software pode ter seus erros conhecidos registrados.

Novos erros podem, porém, ser detectados pelos usuários ao longo do uso do software. Quando relatados, tais erros devem ser incluídos no relatório de erros da versão do software e, dependendo de como a disciplina é organizada, imediatamente encaminhados ao setor de manutenção para que o defeito seja identificado e corrigido. O processo deve ser rastreado de forma que se saiba sempre se o erro foi encaminhado, se já foi resolvido e se foi incorporado a uma nova versão do software. Além disso, deve-se saber se o usuário que relatou o erro recebeu a nova versão do software em um prazo razoável.

14.2.2 MANUTENÇÃO ADAPTATIVA

Conforme visto nas Leis de Lehman ([Seção 14.1](#)), a manutenção adaptativa é inevitável quando se trata de sistemas de software que evoluem. Isso porque:

- Requisitos de cliente e usuário mudam com o passar do tempo.

- Novos requisitos surgem.
- Leis e normas mudam.
- Tecnologias novas entram em uso etc.

O sistema desenvolvido poderá estar ou não preparado para acomodar tais modificações de contexto. Na verdade, qualquer requisito inicialmente identificado é passível de mudança durante a operação do sistema. Cabe ao analista, juntamente com os outros interessados, na fase de levantamento de requisitos, identificar quais desses requisitos serão considerados permanentes e quais serão considerados transitórios (Wazlawick, 2015). Os transitórios podem ser adaptados por um simples processo de configuração pelo usuário do produto. Já os permanentes somente serão alterados a partir de um processo de manutenção, que pode envolver inclusive a refatoração do software.

Considere, por exemplo, que um sistema de venda de livros (Livraria Virtual) será produzido e um dos requisitos suplementares é de que a moeda usada é o Real. Se o analista considerar esse requisito como transitório, deverá implementar no sistema a possibilidade de trabalhar com outras moedas, mesmo que em um primeiro momento apenas o Real seja cadastrado. Se novas moedas passarem a ser utilizadas no futuro, bastará cadastrá-las.

Mas esse processo tem um custo de desenvolvimento mais alto do que se o analista considerar o requisito como permanente. O fato de o requisito ser considerado *permanente* significa que o sistema não será preparado para a sua mudança. Assim, apenas uma moeda existirá para o sistema (o Real), e não será possível cadastrar novas moedas. Se no futuro surgir a necessidade de cadastrar uma nova moeda, será necessário aplicar manutenção adaptativa ao sistema, refatorando algumas de suas partes para passar a trabalhar com essa nova moeda. Assim, a economia que se obteve ao longo do projeto será gasta durante a operação do sistema.

Com os *requisitos permanentes* acontece o seguinte:

- É mais barato e rápido incorporá-los ao software durante o desenvolvimento.
- É mais caro e demorado mudá-los depois que o software está em operação.

Com os *requisitos transitórios* acontece o inverso:

- É mais caro e demorado incorporá-los ao software durante o desenvolvimento.
- É mais barato e rápido mudá-los depois que o software está em operação.

Dessa forma, pode-se considerar que o analista deve ponderar, antes de decidir se um requisito é transitório ou permanente, qual é a real probabilidade de que ele mude durante a operação do sistema, qual é o impacto (custo) de se prevenir para essa mudança ao longo do projeto e qual é o impacto de mudar o sistema durante sua operação. Ponderando esses três valores, o analista, juntamente com o cliente, deverá tomar uma decisão. De um lado da balança estará o custo de implementar a flexibilidade do requisito no projeto e do outro lado da balança o custo de acomodar a mudança do requisito depois de o sistema estar em operação combinado com a probabilidade de o requisito realmente mudar.

Em geral, até pelo princípio de simplicidade dos modelos ágeis, na falta de bons motivos para o contrário, a maioria dos requisitos são definidos como permanentes.

14.2.3 MANUTENÇÃO PERFECTIVA

A *manutenção perfectiva* consiste em mudanças que afetam mais as características de desempenho do que as características de funcionalidade do software. Em geral, tais melhorias são buscadas em função da pressão de mercado, visto que se preferem produtos mais eficientes ou com melhor usabilidade e com mesma funcionalidade a produtos menos eficientes, especialmente em áreas nas quais o processamento é crítico, como jogos e sistemas de controle em tempo real.

A melhoria de características vai estar quase sempre ligada às qualidades externas do software (Capítulo 11), mas especialmente às qualidades ligadas a segurança, confiabilidade, usabilidade e eficiência de desempenho.

14.2.4 MANUTENÇÃO PREVENTIVA

A *manutenção preventiva* pode ser realizada através de atividades de reengenharia, nas quais o software é modificado para resolver problemas potenciais.

Um sistema que suporta até cinquenta acessos simultâneos e que já conta com picos de vinte a trinta acessos, por exemplo, pode sofrer um processo de manutenção preventiva através de reengenharia ou refatoração de sua arquitetura, de forma que passe a suportar quinhentos acessos, afastando a possibilidade de colapso por um período de tempo razoável.

Outro uso da manutenção preventiva consiste em aplicar técnicas de engenharia reversa como refatoração ou redocumentação para melhorar a manutenção do software (Seção 14.6).

14.3 Processo de Manutenção

A manutenção pode ser vista como um processo a ser executado. A não ser, normalmente, no caso da manutenção preventiva, esse processo costuma ser iniciado a partir de uma requisição do cliente ou usuário do software.

No caso da manutenção corretiva, pode ser interessante que o usuário apresente uma descrição clara do erro, que pode ser produzida a partir da captura de telas ou *log* de sistema.

No caso da manutenção perfectiva, o usuário deve descrever da melhor forma possível os novos requisitos que deseja. Para efetuar essa atividade, ele pode precisar do apoio de um analista capacitado em levantamento de requisitos.

Sugere-se que as tarefas de manutenção identificadas sejam priorizadas e colocadas em uma fila de prioridades para serem resolvidas de acordo com a urgência. Nessa fila, as solicitações de manutenção perfectiva costumam ocupar os últimos lugares. No caso da manutenção corretiva e adaptativa, o impacto do erro ou inadequação sobre os usuários deve ser o primeiro critério de desempate, devendo ser tratados prioritariamente os problemas de maior impacto.

Recomenda-se também que, uma vez selecionada uma tarefa de manutenção, as seguintes ações sejam efetuadas:

- Análise de esforço para a tarefa de manutenção (Seção 14.5).
- Análise de risco para a tarefa de manutenção, verificando possíveis riscos, sua probabilidade e impacto, elaborando e executando possíveis planos de mitigação (Capítulo 8).
- Planejamento da tarefa de manutenção, estabelecendo prazos, responsáveis, recursos e entregas (Capítulo 6).
- Execução da tarefa de manutenção.

Ao final da tarefa de manutenção devem ser executados os testes de regressão (Seção 13.2.6).

A Norma ISO/IEC 14764:2006 define que o processo de manutenção de software compreende seis atividades:

- Implementação do processo.
- Análise do problema e modificação.
- Implementação da modificação.
- Revisão da manutenção/aceitação.
- Migração.
- Desativação (*retirement*).

A **Figura 14.1** apresenta de forma esquemática as relações entre estas atividades.

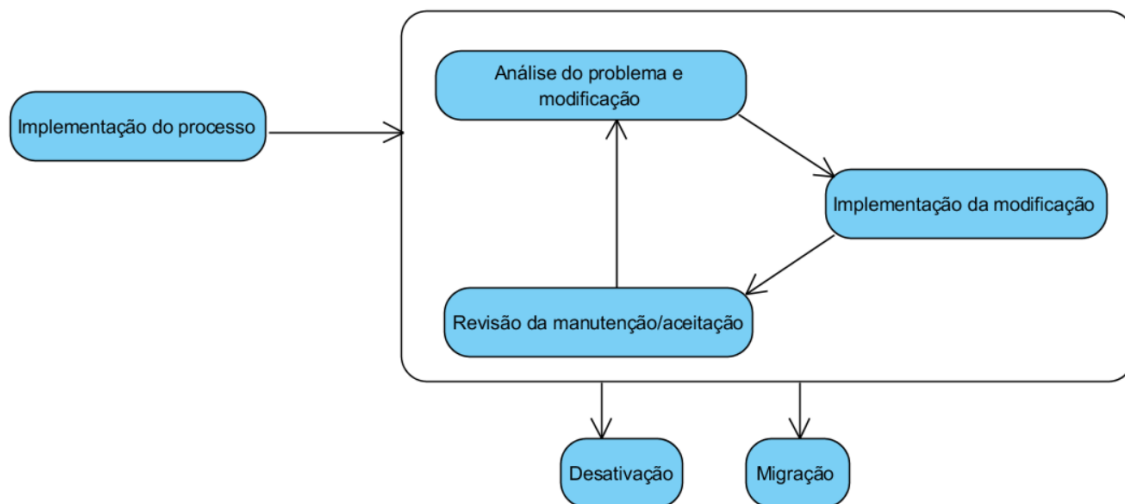


Figura 14.1 Relação entre as atividades de manutenção (Fonte: ISO/IEC 14764:2006)

Durante a *implementação do processo*, o responsável pela manutenção vai estabelecer os planos e procedimentos que serão executados durante a manutenção. A principal saída desta atividade é o plano de manutenção.

Durante a *análise do problema e modificação*, o analista deve:

- Analisar a solicitação de modificação ou relato de problema.
- Replicar ou verificar o problema.
- Desenvolver opções para implementar a modificação.
- Documentar a solicitação de modificação, relato de problema, os resultados da análise e opções de implementação.
- Obter aprovação para a opção de modificação selecionada.

Durante a *implementação da modificação* o responsável pela manutenção desenvolve e testa a modificação no produto de software.

A atividade de *revisão da manutenção/aceitação* deve assegurar que as modificações feitas no sistema estão corretas e de acordo com padrões aprovados.

Durante a vida de um sistema, ele pode, eventualmente, ter que ser migrado para ambientes diferentes. Para que isso seja feito, o responsável pela manutenção deve determinar as ações necessárias para a migração e, então, desenvolver e documentar os passos necessários para efetivá-la, constituindo, assim, o plano de migração.

Uma vez que um produto de software tenha atingido o que é considerado o final de sua vida útil, ele deve ser desativado. Uma análise especialmente baseada em critérios econômicos deve ser feita para determinar o plano de desativação. Uma análise deve ser feita para determinar se é economicamente interessante:

- Manter tecnologia desatualizada.
- Mudar para uma nova tecnologia através do desenvolvimento de um novo produto de software.
- Desenvolver um novo produto de software para obter modularidade.
- Desenvolver um novo produto de software para facilitar a manutenção.

- Desenvolver um novo produto de software para obter padronização.
- Desenvolver um novo produto de software para facilitar a independência do vendedor.

Normalmente um produto de software desativado é substituído por outro. Mas nem sempre é o caso. No plano de desativação, o responsável pela manutenção deve ter especial preocupação em relação a como os dados legados serão acessados ou convertidos.

14.4 Tipos de Atividades de Manutenção e suas Métricas

Vários autores, pelo menos desde Boehm (1981), apontam que as atividades de manutenção, longe de serem um mero detalhe, são aquelas em que as empresas colocam mais esforço.

A análise de centenas de projetos em longo prazo mostrou que foram colocados mais tempo e mais esforço nas atividades de manutenção do que nas atividades de desenvolvimento de software. Assim, é necessário que se tenha também modelos de estimação de esforço e custo para atividades de manutenção.

Jones (2002) apresenta um estudo em que se observa que, em média, cada funcionário remove oito defeitos de um sistema em operação por mês. Porém, as atividades de manutenção variam muito e podem ser classificadas em vários tipos (Jones, 1998), os quais serão discutidos nas próximas subseções.

14.4.1 REPARAÇÃO DE DEFEITOS

A *reparação de defeitos* é, possivelmente, a atividade mais importante e urgente em manutenção de software, porque se destina a eliminar problemas que não deveriam existir. O custo dessas atividades normalmente é absorvido pela empresa desenvolvedora, a não ser que cláusulas contratuais específicas estabeleçam outro tipo de entendimento.

Difícilmente essas atividades podem ser estimadas com pontos de função, porque a maioria das atividades de reparação de defeitos implica escrever algumas poucas linhas de código, o que equivaleria a uma fração de um ponto de função. Mas, de outro lado, algumas dessas atividades podem ser altamente consumidoras de tempo.

Uma métrica mais comum para esse tipo de atividade, uma vez que os custos são arcados pela organização desenvolvedora, é o número de defeitos que a organização consegue reparar em um mês. Um valor aceitável, de acordo com normas norte-americanas, é de 8 defeitos reparados por mês. Porém, empresas com bons processos e práticas conseguem reparar até 20 defeitos por mês em seus sistemas.

Jones (1998) ainda indica que, conforme a gravidade do erro, diferentes tempos de espera são toleráveis (Tabela 14.1).

TABELA 14.1 Tempo de resposta ao erro em função de sua gravidade (Fonte: Jones, 1998)

Gravidade	Significado	Tempo nominal da descoberta aos reparos iniciais	Percentual em relação aos defeitos relatados
1	Aplicação não funciona	1 dia	1%
2	Funcionalidade principal não funciona	2 dias	12%
3	Funcionalidade	30 dias	52%

Gravidade	Significado	Tempo nominal da descoberta aos reparos iniciais	Percentual em relação aos defeitos relatados
	secundária não funciona		
4	Erro cosmético	120 dias	35%

Os fatores a seguir ainda podem influenciar a estimativa de esforço a ser aplicada às atividades de reparação de defeitos:

- *Defeitos suspensos (abeyant)*: em cerca de 10% das vezes, a falha relatada pelo cliente não é reproduzida no ambiente de manutenção. É a típica situação “na minha máquina funciona”. Esse tipo de defeito deve-se a combinações de condições (versão do sistema operacional, outros produtos instalados na mesma máquina etc.) que, muitas vezes, são difíceis de detectar e reproduzir, portanto é o tipo mais caro de manutenção corretiva. Esses defeitos ficam suspensos até que se consiga repeti-los.
- *Defeitos inválidos*: cerca de 15% dos defeitos relatados por usuários não são propriamente defeitos no software, mas produto de erros gerados pelos próprios usuários ou por sistemas relacionados. Mesmo assim, esses problemas devem ser catalogados e processados, e sua análise demanda tempo e esforço da empresa que faz a manutenção do sistema.
- *Consertos ruins (bad fix injection)*: cerca de 7% das atividades de correção de erros acabam introduzindo novos erros no software. Essa porcentagem pode variar de 1 a 20%, dependendo do nível de qualidade do processo de manutenção e da seriedade com que os testes de regressão são feitos.
- *Defeitos duplicados*: em sistemas com muitos usuários é comum que um mesmo defeito seja relatado por mais de um usuário. Assim, embora o defeito só precise ser resolvido uma vez, o fato de ele ser relatado por vários usuários faz que seja necessário investir tempo nisso. Grandes empresas de software chegam a ter 10% de seus relatos de defeitos classificados como defeitos duplicados.

Frequentemente é difícil identificar defeitos duplicados, pois o relato de um para outro usuário pode variar, mesmo que estejam descrevendo os efeitos de um mesmo defeito originalmente.

14.4.2 REMOÇÃO DE MÓDULOS SUJEITOS A ERROS

Uma pesquisa realizada pela IBM nos anos 1960 demonstrou que os defeitos não se distribuem aleatoriamente ao longo de uma aplicação. Muito pelo contrário: eles tendem a se concentrar em determinados módulos da aplicação. Foi observado que, em um grande sistema da empresa, com 425 módulos, 300 módulos nunca foram alvo de manutenção corretiva, enquanto outros 31 módulos concentraram cerca de 2.000 relatos de erros ao longo de um ano, correspondendo a mais de 60% do total de erros relatados para o produto inteiro (Mukhija, 2003).

Módulos sujeitos a defeitos podem nunca estabilizar, porque a taxa de consertos ruins pode passar de 100%, ou seja, a cada defeito consertado, novos defeitos podem acabar sendo introduzidos¹

¹A internet apresenta uma brincadeira relacionada a esse fato, que é incorporada à “técnica” XGH, ou *eXtreme Go Horse*. Uma das regras dessa técnica estabelece que, a cada defeito consertado com XGH, sete novos defeitos sejam criados, o que faz a quantidade de defeitos tender ao infinito.

14.4.3 SUPORTE A USUÁRIOS

O *suporte a usuários* fará a interface entre o cliente do software e a empresa que presta manutenção ao software. O suporte a usuários costuma receber as reclamações, fazer uma triagem delas, encaminhar uma solução previamente conhecida ao cliente ou o problema ao setor de manutenção.

O tamanho da equipe de suporte dependerá de vários fatores, entre os quais os mais importantes são a quantidade esperada de defeitos e a quantidade de clientes.

Estima-se que, para um software típico (que não apresenta grandes problemas de qualidade logo de partida), um atendente consiga tratar as chamadas de cerca de 150 clientes por mês, caso o meio de contato seja o telefone. De outro lado, se o meio de contato for *e-mail* ou *chat*, esse número pode subir para mil usuários por atendente por mês.

14.4.4 MIGRAÇÃO ENTRE PLATAFORMAS

A *migração* de um produto para outra plataforma, quando se trata de software personalizado, é feita por demanda do cliente. Quando se trata de um software de prateleira, ela é feita com a intenção de aumentar o mercado.

Normalmente, migrações são projetos por si só, embora possam ser consideradas atividades de evolução de software. Assume-se que sistemas desenvolvidos de acordo com boas práticas e com boa documentação possam ser migrados a uma taxa de cinquenta pontos de função por desenvolvedor-mês. Porém, se os sistemas forem mal documentados e tiverem organização obscura, essa taxa pode baixar para até cinco pontos de função por desenvolvedor-mês.

14.4.5 CONVERSÃO DE ARQUITETURA

Geralmente, uma *conversão de arquitetura* de sistema é feita por pressão tecnológica. É o caso, por exemplo, de mudar de arquivos simples para bancos de dados relacionais, ou de mudar uma interface orientada a linha de comando para uma interface gráfica.

No caso da migração entre plataformas, se o software for personalizado, a conversão possivelmente será uma demanda do cliente, enquanto no caso de software de prateleira será uma estratégia para buscar novos mercados.

A conversão de arquitetura também pode ser uma estratégia para melhorar a manutenção de um sistema, em que se pode transformar um sistema monolítico ou feito com blocos *ad hoc* em um sistema bem estruturado com componentes e classes coesas.

A produtividade de um projeto de conversão de arquitetura dependerá basicamente da qualidade das especificações do sistema. Quanto mais obscuras forem as especificações, mais difícil será a conversão.

Em geral, sistemas mal documentados ou obscuros precisam passar por processos de engenharia reversa antes de serem convertidos para uma nova arquitetura.

14.4.6 ADAPTAÇÕES OBRIGATÓRIAS

Talvez os piores tipos de manutenção de software sejam as *adaptações obrigatórias*, decorrentes de mudanças em normas e leis, formas de cálculo de impostos etc. O problema é que essas mudanças não podem ser previstas pela equipe de desenvolvimento ou manutenção, nem mesmo pelo cliente. Além disso, normalmente têm um prazo curto e estrito para serem aplicadas, e as penalidades por não adaptação costumam ser altas.

14.4.7 OTIMIZAÇÃO DE PERFORMANCE

Atividades de *otimização de performance* implicam analisar e resolver gargalos da aplicação, geralmente relacionados com o acesso a dados, processamento e número de usuários simultâneos.

Essas atividades variam muito em relação ao tipo e à carga de trabalho, por isso é muito difícil estabelecer um padrão para a estimativa de custos. Uma técnica que pode ser empregada em alguns casos é a otimização estilo *anytime*, usando *timeboxing*, ou seja, faz-se a melhor otimização possível dentro do tempo e com os recursos previamente destinados a essa atividade.

14.4.8 MELHORIAS

As *melhorias* são um tipo de manutenção adaptativa e perfectiva normalmente iniciado por solicitação dos clientes, que em geral acabam arcando com os custos relacionados.

Muitas vezes, elas implicam a introdução de novas funcionalidades, de forma que possam ser aplicadas técnicas usuais de estimação de esforço por CII, pontos de função ou pontos de caso de uso.

Pode-se considerar que existem dois tipos de melhoria:

- *Pequenas melhorias*: consistem de aproximadamente cinco pontos de função, ou seja, da introdução de um novo relatório, consulta ou tela.
- *Grandes melhorias*: consistem de um número significativamente maior de pontos de função, em geral com mais de vinte pontos de função, e devem ser tratadas como pequenos projetos de desenvolvimento.

Um dos aspectos que possivelmente diferenciam a estimativa de esforço das melhorias em relação ao desenvolvimento do novo software é que no caso das melhorias deve-se levar em conta o estado atual do sistema. Se o sistema for bem organizado e documentado será mais fácil integrar as novas funcionalidades; caso contrário, haverá um esforço de integração maior, que deverá ser levado em conta como fator técnico no momento de aplicar a estimativa.

Estima-se que sistemas em operação, em média, aumentam seus pontos de função em cerca de 7% anualmente em função de melhorias (Jones, 1998).

14.5 Modelos de Estimação de Esforço de Manutenção

Modelos de estimação de esforço para atividades de manutenção foram propostos na literatura, alguns dos quais são apresentados nas subseções seguintes. Assim como os modelos de estimação de esforço de desenvolvimento, esses modelos paramétricos podem usar como base para a estimação tanto o número de pontos de função quanto o número de linhas de código.

14.5.1 MODELO ACT

O modelo *ACT* (*Annual Change Traffic*, ou Tráfego de Mudança Anual) baseia-se em uma estimativa da porcentagem de linhas de código que vão sofrer manutenção. São consideradas linhas em manutenção tanto as linhas de código novas criadas quanto as linhas alteradas durante a manutenção. O valor da variável *ACT* é, portanto, o número de linhas que sofrem manutenção dividido pelo número total de linhas do código em um ano típico.

Boehm (1981) estabeleceu a seguinte equação para calcular o esforço estimado de manutenção durante um ano:

$$E = ACT \times SDT$$

Nesta equação, *E* é o esforço, medido em desenvolvedor-mês, a ser aplicado no período de um ano nas atividades de manutenção, *ACT* é a porcentagem esperada de linhas modificadas ou adicionadas durante um ano em relação ao tamanho do software, e *SDT* é o tempo de desenvolvimento do

software (*Software Development Time*) também em desenvolvedor-mês.

Um software que foi desenvolvido com um esforço de oitenta desenvolvedores-mês, por exemplo, terá $SDT = 80$. Se a taxa anual esperada de linhas em manutenção (ACT) for de 2%, então o esforço anual esperado de manutenção para esse software será dado por:

$$E = 0,02 \times 80 = 1,6$$

Assim, para esse sistema hipotético, de acordo com essa fórmula, espera-se um esforço anual de 1,6 desenvolvedor-mês em atividades de manutenção em cada um dos anos seguintes.

Schaefer (1985) apresenta a seguinte variação para essa fórmula:

$$E = ACT \times 2,4 \times KSLOC^{1,05}$$

Ou seja, Schaefer substitui o tempo de desenvolvimento do produto (caso ele não seja conhecido) por uma fórmula baseada no número total de milhares de linhas de código do produto ($KSLOC$). Assim, um software com 20 mil linhas de código e ACT de 2% teria o seguinte esforço anual de manutenção (em desenvolvedor-mês):

$$E = 0,02 \times 2,4 \times 20^{1,05} = 1,115$$

Esse modelo tem as mesmas desvantagens do modelo COCOMO 81, ou seja, não é realisticamente aplicável a sistemas novos quando não existem dados históricos para ACT , a quantidade de linhas de código modificadas não necessariamente indica esforço de manutenção e, sobretudo, a abordagem não usa nenhum atributo das atividades de manutenção como base para calcular o esforço. Porém, na falta de outras informações, é um método simples de aplicar.

14.5.2 MODELO DE MANUTENÇÃO DE CII

Como já foi visto na Seção 7.4, CII é um método de estimação de esforço de desenvolvimento que toma como entrada o número de linhas de código a serem desenvolvidas ou pontos de função convertidos em $KSLOC$.

A equação de estimação de esforço para manutenção é semelhante à equação usada no modelo *post-architecture*, com a seguinte forma:

$$E = A * KSLOC_m^S * \prod_{i=1}^n M_i$$

Nesta equação:

- E é o esforço de manutenção em desenvolvedor-mês a ser calculado.
- A é uma constante calibrada pelo método, inicialmente valendo 2,94.
- $KSLOC_m$ é o número de linhas de código que se espera adicionar ou alterar, ajustado pelo fator de manutenção (ver a seguir). Não são contadas as linhas que eventualmente serão excluídas do código.
- S é o coeficiente de escala determinado pelos fatores de escala e calculado como mostrado na Seção 7.4.3.
- M_i são os multiplicadores de esforço.

As seguintes mudanças em relação ao modelo *post-architecture* são implementadas para o cálculo do esforço de manutenção:

- O multiplicador de esforço SCED (cronograma de desenvolvimento requerido) não é usado (ou assumido como nominal), porque se espera que os ciclos de manutenção tenham duração fixa predeterminada.
- O multiplicador de esforço RUSE (desenvolvimento para reuso) não é usado (ou assumido como nominal), porque se considera que o esforço requerido para manter a reusabilidade de

um componente de software é balanceado pela redução do esforço de manutenção por causa do projeto, da documentação e dos testes cuidadosos do componente.

- O multiplicador de esforço RELY (software com confiabilidade requerida) tem uma tabela de aplicação diferenciada. Assume-se que RELY na fase de manutenção vai depender do valor que RELY tinha na fase de desenvolvimento. Se o produto foi desenvolvido com baixa confiabilidade, haverá maior esforço para consertá-lo. Se o produto foi desenvolvido com alta confiabilidade, haverá menor esforço para consertá-lo (Tabela 14.2), exceto no caso de sistemas com risco à vida humana, nos quais a necessidade de confiabilidade, mesmo na fase de manutenção, faz crescer o esforço.

TABELA 14.2 Forma de obtenção do equivalente numérico para RELY na fase de manutenção

Descritor	Pequena inconveniência	Perdas pequenas facilmente recuperáveis	Perdas moderadas facilmente recuperáveis	Alta perda financeira	Risco à vida humana	
Avaliação	Muito baixo	Baixo	Nominal	Alto	Muito alto	Extra alto
Equivalente numérico	1,23	1,10	1,00	0,99	1,07	n/a

Assim, a tabela dada deve ser aplicada da seguinte forma: avalia-se a confiabilidade com a qual o sistema foi originalmente desenvolvido (primeira linha) e encontra-se o valor numérico a ser aplicado na fase de manutenção na terceira linha. Note que os valores são inversamente proporcionais aos usados na fase de desenvolvimento (Tabela 7.17).

O número de KSLOC usado na fase de manutenção deve ser ajustado antes de ser aplicado na equação de cálculo de esforço pelo uso do *fator de ajuste de manutenção (MAF)*:

$$KSLOC_m = (KSLOC_{adicionadas} + KSLOC_{modificadas}) * MAF$$

O fator de ajuste de manutenção *MAF* é calculado a partir da equação a seguir:

$$MAF = 1 + \left(\frac{SU}{100} * UNFM \right)$$

Nesta equação:

- *SU* é o fator de ajuste relacionado à *compreensão do software (software understanding)*, calculado de acordo com a Tabela 14.3.
- *UNFM* é o fator de *não familiaridade* com relação ao software, calculado de acordo com a Tabela 14.4.

TABELA 14.3 Forma de cálculo do equivalente numérico para *SU*

Descritor	Muito baixo	Baixo	Nominal	Alto	Muito alto
Estrutura	Coesão muito baixa, acoplamento alto, código espaguete	Coesão moderadamente baixa, acoplamento alto	Razoavelmente bem estruturado, algumas áreas fracas	Alta coesão, baixo acoplamento	Modularidade forte, ocultamento de informação em estruturas de dados ou controle (objetos)
Clareza da	As visões do	Alguma correlação	Correlação	Boa	As visões do

Descritor	Muito baixo	Baixo	Nominal	Alto	Muito alto
aplicação	programa e sua aplicação no mundo real não batem	entre o programa e a aplicação	moderada entre o programa e a aplicação	correlação entre o programa e a aplicação	programa e da aplicação no mundo real claramente batem
Autodescrição	Código obscuro, documentação faltando, obscura ou obsoleta	Alguns comentários no código e cabeçalhos, alguma documentação útil	Nível moderado de documentação no código e cabeçalhos	Código e cabeçalhos bem documentado, algumas áreas fracas	Código autodescritivo, documentação atualizada e bem organizada baseada em <i>design</i>
Valor de SU	50	40	30	20	10

TABELA 14.4 Forma de cálculo do equivalente numérico para *UNFM*

Nível de não familiaridade	Valor de UNFM
Completamente familiar	0,0
Basicamente familiar	0,2
Um tanto familiar	0,4
Um tanto não familiar	0,6
Basicamente não familiar	0,8
Completamente não familiar	1,0

Caso não se conheça *a priori* o número de linhas a serem adicionadas ou modificadas, esse valor pode ser obtido a partir de uma análise histórica dos processos de manutenção do projeto ou da empresa. A partir de uma estimativa da porcentagem de linhas adicionadas ou alteradas, pode-se prever a quantidade de manutenção que será necessária no futuro.

14.5.3 MODELOS FP E SMPEEM

O modelo *FP* (Albrecht, 1979) para cálculo de esforço de manutenção é baseado unicamente em pontos de função, e não em linhas de código. Segundo esse modelo, é necessário calcular os pontos de função não ajustados de quatro tipos de funções:

- *ADD*: UFP de funções que vão ser adicionadas.
- *CHG*: UFP de funções que vão ser alteradas.
- *DEL*: UFP de funções que vão ser removidas.
- *CFP*: UFP de funções que serão adicionadas por conversão.

Além de classificar as entradas, saídas, consultas, arquivos internos e arquivos externos nesses quatro tipos antes de contabilizar seus pontos de função não ajustados, a técnica propõe que os fatores de ajuste técnico (*VAF*) (Seção 7.1.6) sejam calculados para dois momentos: antes da

manutenção (VAF_A) e depois da manutenção (VAF_D). Isso ocorre porque possivelmente a atividade de manutenção pode alterar um ou mais destes fatores fazendo com que, por exemplo, o sistema fique mais seguro, ou passe a ser distribuído, ou ainda, que apresente mais facilidade para o usuário. A equação seguinte é, então, aplicada:

$$E = (ADD + CHG + CPF) * VAF_D + DEL * VAF_A$$

Ahn, Suh, Kim e Kim (2003) apresentam uma evolução desse modelo, chamada *SMPEEM* (*Software Maintenance Project Effort Estimation Model* ou Modelo de Estimação de Esforço para Projeto de Manutenção de Software), na qual incluem mais dez fatores de ajuste específicos para as atividades de manutenção:

- Conhecimento do domínio da aplicação.
- Familiaridade com a linguagem de programação.
- Experiência com o software básico (sistema operacional, gerenciador de banco de dados).
- Estruturação dos módulos de software.
- Independência entre os módulos de software.
- Legibilidade e modificabilidade da linguagem de programação.
- Reusabilidade de módulos de software legados.
- Atualização da documentação.
- Conformidade com padrões de engenharia de software.
- Testabilidade.

Os três primeiros fatores são referentes às habilidades de engenharia da equipe. Os quatro fatores seguintes são referentes a características técnicas, e os três últimos fatores estão relacionados ao ambiente de manutenção.

14.6 Engenharia Reversa e Reengenharia

Em algumas situações, o processo de manutenção ou evolução de um sistema exige uma atividade mais drástica do que simplesmente consertar partes do código. Sistemas antiquados, mal documentados e mal mantidos poderão requerer um processo completo de reengenharia para que possam voltar a evoluir de modo mais saudável.

A reengenharia de um sistema é, basicamente, o processo de descobrir como um sistema funciona para que se possa refatorá-lo ou mesmo criar um novo sistema tecnologicamente atualizado que cumpra suas tarefas.

Chikofsky e Cross II (1990) apresentam uma taxonomia de termos relacionados à reengenharia de software:

- *Engenharia direta (forward engineering)*: processo tradicional de produção de software que vai das abstrações de mais alto nível até o código executável.
- *Engenharia reversa (reverse engineering)*: processo de analisar um sistema ou seus modelos de forma a conseguir produzir especificações de nível mais alto. É um processo de exame e explicação.
- *Redocumentação (redocumentation)*: é uma subárea da engenharia reversa. Em geral, trata-se de obter formas alternativas de uma especificação no mesmo nível do artefato examinado.
- *Recuperação de projeto (design recovery)*: é outra subárea da engenharia reversa. Ao contrário da anterior, a recuperação de projeto vai realizar abstrações a partir dos elementos examinados, a fim de produzir artefatos em níveis mais altos do que os examinados.
- *Reestruturação (restructuring)*: é uma das formas de refatoração e consiste em transformar

um artefato internamente, porém mantendo sua funcionalidade aparente. Normalmente, a reestruturação é realizada para simplificar a arquitetura de sistemas de forma a minimizar futuros problemas de manutenção.

- *Reengenharia (reengineering)*: exame e alteração de um sistema para reconstruí-lo de forma diferente. Geralmente, inclui alguma forma de engenharia reversa seguida de engenharia direta ou reestruturação.

A Figura 14.2 apresenta, esquematicamente, a relação entre os diferentes termos.

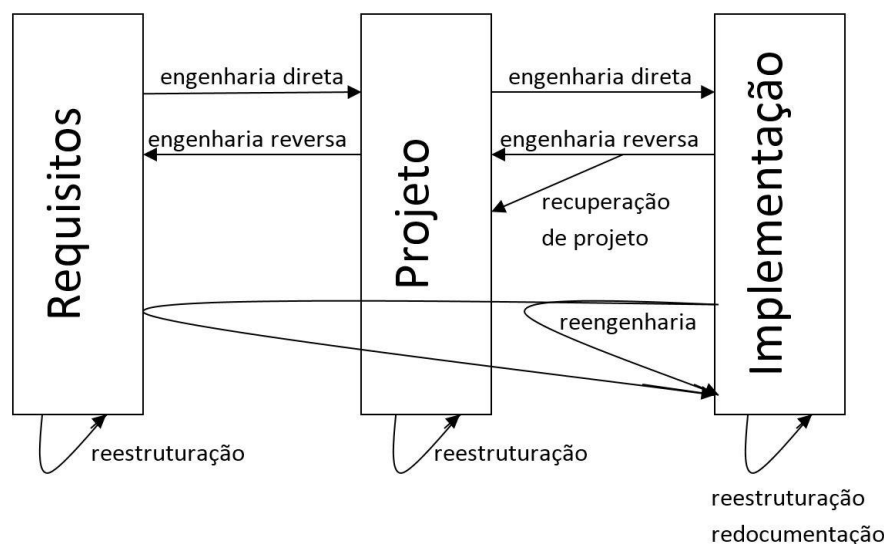


Figura 14.2 Relação esquemática entre os diferentes termos relacionados à engenharia reversa (Fonte: Chikofsky & Cross II, 1990)

Segundo os autores, a expressão “engenharia reversa” teve origem na análise de hardware, em que a prática de decifrar projetos a partir de produtos prontos é usual. Porém, na área de hardware, o objetivo da engenharia reversa normalmente é obter informações para duplicar um produto. Já na área de software os objetivos podem ser outros:

- Lidar com a complexidade do código.
- Gerar visões alternativas, como diagramas.
- Recuperar informações perdidas sobre modificações não documentadas.
- Detectar efeitos colaterais que não foram previstos no processo de engenharia direta.
- Sintetizar conceitos e estrutura em abstrações de nível mais alto.
- Facilitar o reuso de ativos de software existentes (por exemplo, para aproveitá-los em linhas de produto de software).

Uma das diretrizes da engenharia reversa consiste em, por causa de seu alto custo, aplicá-la apenas onde for realmente necessária, não necessariamente ao sistema todo (Bennett, 2000).

Em função do seu objeto, a engenharia reversa pode ser classificada em dois tipos: *engenharia reversa de código* e *engenharia reversa de dados*. As subseções a seguir apresentam esses dois tipos.

14.6.1 ENGENHARIA REVERSA DE CÓDIGO

Segundo Muller et al. (2000), a engenharia reversa de software tem se concentrado no *código*,

enquanto a engenharia direta atua mais amplamente nos diferentes *níveis de abstração* de uma especificação de sistema. Isso pode ser explicado pelo fato de que, em geral, a questão de engenharia reversa se coloca quando existe um sistema legado e é necessário, entre outras coisas, descobrir regras de negócio escondidas nele.

O problema é que muitas vezes o código, por si só, não contém todas as informações de que se necessita. Muitas decisões tomadas ficam registradas apenas na memória dos desenvolvedores. Com o tempo, essas memórias podem ser perdidas ou as pessoas podem ficar inacessíveis. Além disso, segundo as Leis de Lehman (**Seção 14.1**), a complexidade do software tende a crescer com o tempo. Assim, fica cada vez mais difícil lidar com o código.

É possível caracterizar dois tipos de engenharia reversa de código:

- A partir de código-fonte.
- A partir de código-objeto.

O primeiro tipo é usado para melhorar a compreensão sobre sistemas legados ou mal documentados. Já o segundo tipo pode ser usado, entre outras coisas, para a clonagem de produtos. Nesse caso, pode-se usar a técnica de *clean room design* (**Schwartz, 2001**) para evitar a quebra de direitos autorais. Essa técnica consiste em usar a engenharia reversa apenas para entender o produto original e, então, desenvolver um produto novo que não use quaisquer partes do produto original.

A engenharia reversa de código pode ser realizada a partir de uma ou mais entre as técnicas a seguir:

- *Análise de fluxo de dados*: consiste em verificar o comportamento do sistema como uma caixa-preta, ou seja, sem ter conhecimento de sua estrutura interna. A análise do comportamento do sistema pode permitir, então, que um novo sistema seja desenvolvido para ter o mesmo comportamento.
- *Dessassemblagem*: consiste em usar um desassemblador, que converte o código executável em mnemônicos de linguagem *Assembly*.
- *Descompilação*: consiste em usar um descompilador para obter uma aproximação do código original usado para produzir o executável. Os resultados podem variar bastante, pois há questões difíceis de tratar, como a escolha de nomes para variáveis e procedimentos.

A análise de fluxo de dados pode ser feita com ferramentas como *bus analyzers* ou *packet sniffers*. Um *bus analyzer* é uma combinação de software e hardware usada para monitorar e apresentar, em formato adequado, o fluxo de informações que passa por um barramento de dados escolhido. Já os *packet sniffers* usualmente são conectados a redes de computadores (com ou sem fio).

Um tipo especial de desassemblador é o *debugger*, que permite que o código seja executado e alterações de partes do código sejam feitas interativamente.

Os descompiladores são bem mais complexos do que os desassembladores, pois precisam gerar comandos em nível bem mais alto do que estes últimos, que fazem apenas uma tradução direta dos códigos de máquina em mnemônicos. Os descompiladores costumam trabalhar em fases caracterizadas por diferentes componentes:

- *Carregador*: esse componente faz o carregamento do programa e identifica algumas informações básicas, como o tipo de processador para o qual o código foi gerado e o ponto de entrada. Pode chegar até a encontrar o equivalente ao módulo principal de um programa, a partir do qual as inicializações e chamadas são feitas.
- *Desassemblador*: esse componente procura transformar os códigos de máquina carregados por uma representação mnemônica independente de processador.
- *Identificador de expressões idiomáticas*: alguns processadores usam instruções muito

específicas para realizar operações que seriam bem mais simples em uma linguagem independente de tecnologia. Por exemplo, a instrução *xor eax,eax* é usada para atribuir zero ao registrador *eax*. Ela poderia ser descrita mais claramente como *eax = 0*. Expressões idiomáticas são catalogadas para cada processador.

- *Análise de programa*: o analisador de programa vai identificar sequências de operações e tentar agrupá-las em comandos. Por exemplo, uma expressão que, em linguagem de alto nível, seria escrita como $x = y + 45 * (z - x) / 2$ seria compilada como uma sequência de operações elementares de adição, multiplicação, subtração e divisão. O analisador de programa deve ser capaz de identificar essa sequência e transformá-la na expressão que possivelmente era a original do programa-fonte.
- *Análise de fluxo de dados*: consiste em detectar as variáveis e seu escopo no programa. O problema é complexo, porque a mesma posição de memória pode ser ocupada por mais de uma variável em momentos diferentes, e uma variável pode ocupar mais de uma posição da memória. A abordagem geral para tratar esse problema é baseada em grafos e foi definida por Kildall (1973).
- *Análise de tipos*: observando as operações (de máquina) efetuadas sobre determinadas variáveis, pode-se inferir seu possível tipo. Por exemplo, operações AND nunca são executadas em variáveis de ponto flutuante ou ponteiros.
- *Estruturação*: consiste em transformar estruturas de máquina em estruturas de alto nível, como *if* e *while*.
- *Geração de código*: a fase final consiste em gerar o código na linguagem-alvo. Possivelmente, vários problemas ainda restarão e terão que ser resolvidos interativamente pelo usuário.

14.6.2 ENGENHARIA REVERSA DE DADOS

Engenharia reversa de dados pode ser considerada um caso especial da engenharia reversa cujo foco está na localização, na organização e na reinterpretação do significado dos dados de um sistema. Davis e Aiken (2000) apresentam um apanhado histórico da evolução dessa área.

Uma das atividades relacionadas à engenharia reversa de dados é a *análise de dados*. Segundo Muller et al. (2000), essa atividade consiste em recuperar um modelo de dados atualizado (a partir de um sistema em operação), estruturalmente completo e semanticamente anotado. A atividade é particularmente difícil de ser automatizada. A recuperação dos modelos de bancos de dados (quando existem) é relativamente simples, mas em geral essas estruturas não contêm informações semânticas e estruturais completas sobre os dados, que acabam sendo diluídas em código executável e documentação.

Posfácio

O Futuro da Engenharia de Software

A ciência da computação é uma área que evolui muito rapidamente, todos sabem. Assim, o leitor deste livro poderia se perguntar: por quanto tempo estes conhecimentos ainda serão úteis? De fato, a evolução das tecnologias e arquiteturas (hoje a computação já está até nas “nuvens”) tem criado grandes desafios e oportunidades nas atividades de desenvolvimento de software. Com o surgimento de geradores de código automatizados realmente efetivos e a consolidação de padrões de projeto e linguagens de programação de nível cada vez mais alto, as atividades de desenvolvimento, especialmente de programação e de teste de código, têm se transformado.

Apesar disso, projetos continuarão a ser feitos e precisarão ser gerenciados. Seus riscos continuarão a ser examinados e o esforço necessário para desenvolvê-los continuará a ser estimado,

mesmo que esse esforço seja cada vez menor relativamente às funcionalidades implementadas. Além disso, a qualidade, tanto do produto final quanto dos processos de trabalho, continuará sempre a ser buscada. Assim, por mais avançadas que sejam as ferramentas de desenvolvimento e por mais automatizados que sejam os geradores de código, o erro humano sempre estará presente, e por muito tempo ainda o teste de software será uma necessidade, nem que seja apenas para verificar se os requisitos foram perfeitamente compreendidos. Então, embora novas tecnologias, normas e abordagens surjam, os conhecimentos aqui examinados continuarão a ser necessários por tempo indeterminado.

Novas tecnologias vão surgir, assim como maneiras mais eficazes e mais eficientes de desenvolver software. Mas o fator humano continuará sendo o motor sem o qual essas tecnologias e ferramentas nada fazem. Os humanos, que estão à frente desse processo, são os depositários e desenvolvedores desses conhecimentos. Um longo caminho já foi trilhado em termos de criação de conceitos e práticas úteis para desenvolver sistemas. Resta agora aplicar o que já foi descoberto e, a partir daí evoluir ainda mais.

REMISSIVO DO CAPÍTULO

ACT. Consulte tráfego de mudança anual
adaptação, 1, 4, 13
adaptações obrigatórias, 13
annual change traffic. Consulte tráfego de mudança anual
autorregulação, 2, 3
carregador, 20
CII, 13, 15
COCOMO 81, 14
complexidade, 3, 19, 20
conserto ruim, 11
conteúdo funcional, 4
conversão de arquitetura, 12
crescimento contínuo, 5
defeito
 duplicado, 11
 inválido, 11
 suspensão, 11
descompilação, 20
dessa montagem, 20
engenharia
 direta, 18
 reversa, 1, 7, 13, 18, 19, 20, 21
 reversa de código, 19, 20
 reversa de dados, 19, 21
esforço, 1, 3, 8, 10, 11, 13, 14, 15, 16, 17, 21

EUP, 1
evolução, 1, 2, 3, 4, 5, 12, 17, 18, 21
familiaridade, 4, 16, 17
feedback, 3, 4, 5
FP, 17
ISO 14764, 5
KSLOC, 14, 15, 16
Leis de Lehman, 1, 2, 4, 6, 20
manutenção, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18
 adaptativa, 4, 6, 13
 corretiva, 1, 4, 5, 8, 11
 perfectiva, 4, 7, 8
 preventiva, 7, 8
melhorias, 4, 7, 13
métrica, 10
migração, 9, 12
mudança contínua, 5
otimização, 1, 13
OUM, 1
Processo Unificado, 1
qualidade, 1, 4, 5, 11, 12, 13, 22
recuperação de projeto, 18
redocumentação, 18
reengenharia, 18
reestruturação, 18
RELY, 15
reparação de defeitos, 10, 11
requisitos permanentes, 7
requisitos transitórios, 7
risco, 8, 15
RUSE, 15
SCED, 15
SMPEEM, 17
SU, 16
suporte a usuários, 12
tipo-E, 2, 5
tipo-P, 2
tipo-S, 2
tráfego de mudança anual, 14
UFP, 17
UNFM, 16, 17
VAF, 17

